week - 1

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
#create the department tables
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);
#create student table
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    marks DECIMAL(5,2),
    dept_id INT,
    FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
);

# insert records
INSERT INTO Department VALUES (10, 'CSE');
INSERT INTO Department VALUES (20, 'ECE');

INSERT INTO Student
VALUES (101, 'Rahul', 20, 85.50, 10);

INSERT INTO Student
VALUES (102, 'Priya', 21, 90.00, 20);

INSERT INTO Student
VALUES (103, 'Amit', 19, 78.50, 10);

COMMIT;

#display the tables
SELECT * FROM Department;

SELECT * FROM Student;

# Alter command usage #
1. add a new column
ALTER TABLE Student ADD (email VARCHAR2(50));
2.modify a column
ALTER TABLE Student MODIFY (student_name VARCHAR2(100));
3.rename a column
ALTER TABLE Student RENAME COLUMN student_name TO full_name;
4.drop a column
ALTER TABLE Student DROP COLUMN email;
```

```
#DROP command
DROP TABLE Student;

DROP TABLE Department;

week - 2

-- DBMS Experiment: DML Operations

-- 1. Create the Student table
CREATE TABLE Student (
    student_id NUMBER(5) PRIMARY KEY,
    student_name VARCHAR2(30),
    age NUMBER(3),
    marks NUMBER(5,2)
);

-- 2. INSERT operation

INSERT INTO Student VALUES (101, 'Rahul', 20, 85.50);

INSERT INTO Student VALUES (102, 'Priya', 21, 90.00);

INSERT INTO Student VALUES (103, 'Amit', 19, 78.50);

-- Save inserted records

COMMIT;

-- 3. SELECT operation

SELECT * FROM Student;

-- Display specific columns
SELECT student_name, marks FROM Student;

-- Display students scoring above 80
SELECT * FROM Student WHERE marks > 80;

-- 4. UPDATE operation
UPDATE Student SET marks = 95 WHERE student_id = 101;
COMMIT;

-- Display updated records
SELECT * FROM Student;

-- 5. DELETE operation
DELETE FROM Student WHERE student_id = 103;
```

```
COMMIT;

-- Display final records
SELECT * FROM Student;

week - 3

-- 1. Create Department table

CREATE TABLE Department (
    dept_id NUMBER(5) PRIMARY KEY,
    dept_name VARCHAR2(30) UNIQUE
);

-- 2. Create Student table with constraints

CREATE TABLE Student (
    student_id NUMBER(5) PRIMARY KEY,
    student_name VARCHAR2(30) NOT NULL,
    age NUMBER(3) CHECK (age >= 18),
    marks NUMBER(5,2) DEFAULT 0,
    dept_id NUMBER(5),
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

-- 3. Insert records into Department

INSERT INTO Department VALUES (10, 'CSE');
INSERT INTO Department VALUES (20, 'ECE');
INSERT INTO Department VALUES (30, 'EEE');

-- 4. Insert records into Student

INSERT INTO Student VALUES (101, 'Rahul', 20, 85, 10);
INSERT INTO Student VALUES (102, 'Priya', 21, 90, 20);
INSERT INTO Student VALUES (103, 'Amit', 19, 75, 10);
INSERT INTO Student VALUES (104, 'Sneha', 22, 95, 30);
INSERT INTO Student VALUES (105, 'Kiran', 20, 80, 20);
COMMIT;

-- 5. ANY operator

SELECT student_name, marks
FROM Student
WHERE marks > ANY (
    SELECT marks FROM Student WHERE dept_id = 10
);

-- 6. ALL operator
```

```sql
SELECT student_name, marks
FROM Student
WHERE marks > ALL (
    SELECT marks FROM Student WHERE dept_id = 10
);

-- 7. IN operator

SELECT student_name
FROM Student
WHERE dept_id IN (10, 20);

-- 8. EXISTS operator

SELECT dept_name
FROM Department D
WHERE EXISTS (
    SELECT 1 FROM Student S
    WHERE S.dept_id = D.dept_id
);

-- 9. NOT EXISTS operator

SELECT dept_name
FROM Department D
WHERE NOT EXISTS (
    SELECT 1 FROM Student S
    WHERE S.dept_id = D.dept_id
);

-- 10. UNION operator

SELECT dept_id FROM Student WHERE marks >= 90
UNION
SELECT dept_id FROM Student WHERE marks < 80;

-- 11. INTERSECT operator

SELECT dept_id FROM Student WHERE marks >= 80
INTERSECT
SELECT dept_id FROM Student WHERE marks < 90;

-- 12. MINUS operator (Oracle equivalent of EXCEPT)

SELECT dept_id FROM Department
MINUS
SELECT dept_id FROM Student;
```

```sql
-- 13. Display all Student records

SELECT * FROM Student;

-- 14. Display all Department records

SELECT * FROM Department;
```

week - 4

```sql
-- 1. Create Department table
CREATE TABLE Department (
    dept_id NUMBER(5) PRIMARY KEY,
    dept_name VARCHAR2(30)
);

-- 2. Create Student table
CREATE TABLE Student (
    student_id NUMBER(5) PRIMARY KEY,
    student_name VARCHAR2(30),
    marks NUMBER(5,2),
    dept_id NUMBER(5),
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

-- 3. Insert records into Department
INSERT INTO Department VALUES (10, 'CSE');
INSERT INTO Department VALUES (20, 'ECE');
INSERT INTO Department VALUES (30, 'EEE');
INSERT INTO Department VALUES (40, 'MECH');

-- 4. Insert records into Student
INSERT INTO Student VALUES (101, 'Rahul', 85, 10);
INSERT INTO Student VALUES (102, 'Priya', 90, 20);
INSERT INTO Student VALUES (103, 'Amit', 75, 10);
INSERT INTO Student VALUES (104, 'Sneha', 95, 30);
INSERT INTO Student VALUES (105, 'Kiran', 80, 20);

COMMIT;

-- 5. Nested Subquery
-- Students scoring above the average marks
SELECT student_name, marks
FROM Student
WHERE marks > (
    SELECT AVG(marks)
    FROM Student
);
```

```sql
-- 6. Correlated Subquery
-- Students scoring above their department's average
SELECT S.student_name, S.marks, S.dept_id
FROM Student S
WHERE S.marks > (
    SELECT AVG(S2.marks)
    FROM Student S2
    WHERE S2.dept_id = S.dept_id
);

-- 7. INNER JOIN
-- Display students with their department names
SELECT S.student_id, S.student_name, D.dept_name
FROM Student S
INNER JOIN Department D
ON S.dept_id = D.dept_id;

-- 8. LEFT OUTER JOIN
-- Display all students, including students without a department
SELECT S.student_id, S.student_name, D.dept_name
FROM Student S
LEFT OUTER JOIN Department D
ON S.dept_id = D.dept_id;

-- 9. RIGHT OUTER JOIN
-- Display all departments, including departments without students
SELECT S.student_id, S.student_name, D.dept_name
FROM Student S
RIGHT OUTER JOIN Department D
ON S.dept_id = D.dept_id;

-- 10. FULL OUTER JOIN
-- Display all students and all departments
SELECT S.student_id, S.student_name, D.dept_name
FROM Student S
FULL OUTER JOIN Department D
ON S.dept_id = D.dept_id;

COMMIT;
```

week - 5

```sql
-- 1. Create Department table
CREATE TABLE Department (
    dept_id NUMBER(5) PRIMARY KEY,
    dept_name VARCHAR2(30)
);

-- 2. Create Student table
CREATE TABLE Student (
```

```
    student_id NUMBER(5) PRIMARY KEY,
    student_name VARCHAR2(30),
    marks NUMBER(5,2),
    dept_id NUMBER(5),
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

-- 3. Insert records into Department
INSERT INTO Department VALUES (10, 'CSE');
INSERT INTO Department VALUES (20, 'ECE');
INSERT INTO Department VALUES (30, 'EEE');

-- 4. Insert records into Student
INSERT INTO Student VALUES (101, 'Rahul', 85, 10);
INSERT INTO Student VALUES (102, 'Priya', 90, 20);
INSERT INTO Student VALUES (103, 'Amit', 75, 10);
INSERT INTO Student VALUES (104, 'Sneha', 95, 30);
INSERT INTO Student VALUES (105, 'Kiran', 80, 20);

COMMIT;

-- 5. COUNT: Count total students
SELECT COUNT(*) AS total_students FROM Student;

-- 6. SUM: Calculate total marks
SELECT SUM(marks) AS total_marks FROM Student;

-- 7. AVG: Calculate average marks
SELECT AVG(marks) AS average_marks FROM Student;

-- 8. MAX: Find highest marks
SELECT MAX(marks) AS highest_marks FROM Student;

-- 9. MIN: Find lowest marks
SELECT MIN(marks) AS lowest_marks FROM Student;

-- 10. GROUP BY: Average marks in each department
SELECT dept_id, AVG(marks) AS average_marks
FROM Student
GROUP BY dept_id;

-- 11. GROUP BY with department names
SELECT D.dept_name, COUNT(S.student_id) AS total_students
FROM Department D
LEFT JOIN Student S
ON D.dept_id = S.dept_id
GROUP BY D.dept_name;

-- 12. HAVING: Departments with average marks above 80
```

```
SELECT dept_id, AVG(marks) AS average_marks
FROM Student
GROUP BY dept_id
HAVING AVG(marks) > 80;

-- 13. CREATE VIEW
CREATE VIEW Student_View AS
SELECT student_id, student_name, marks
FROM Student;

-- 14. Display the view
SELECT * FROM Student_View;

-- 15. Create a view with a condition
CREATE VIEW High_Marks_View AS
SELECT student_id, student_name, marks
FROM Student
WHERE marks >= 85;

-- 16. Display high marks view
SELECT * FROM High_Marks_View;

-- 17. Replace an existing view
CREATE OR REPLACE VIEW Student_View AS
SELECT student_id, student_name, marks, dept_id
FROM Student;

-- 18. Update a record through a simple view
UPDATE Student_View
SET marks = 88
WHERE student_id = 101;

COMMIT;

-- 19. Verify the update
SELECT * FROM Student_View;

-- 20. Drop the views
DROP VIEW High_Marks_View;
DROP VIEW Student_View;

week - 6

-- 1. Create Student table
CREATE TABLE Student (
    student_id NUMBER(5) PRIMARY KEY,
    student_name VARCHAR2(30),
    marks NUMBER(5,2)
);
```

```
-- 2. Create Log table to store trigger actions
CREATE TABLE Student_Log (
    log_id NUMBER(5),
    action_type VARCHAR2(10),
    student_id NUMBER(5),
    old_marks NUMBER(5,2),
    new_marks NUMBER(5,2),
    log_date DATE
);

-- 3. Create sequence for log IDs
CREATE SEQUENCE log_seq
START WITH 1
INCREMENT BY 1;

-- 4. INSERT Trigger
CREATE OR REPLACE TRIGGER trg_student_insert
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Log
    VALUES (
        log_seq.NEXTVAL,
        'INSERT',
        :NEW.student_id,
        NULL,
        :NEW.marks,
        SYSDATE
    );
END;
/

-- 5. UPDATE Trigger
CREATE OR REPLACE TRIGGER trg_student_update
AFTER UPDATE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Log
    VALUES (
        log_seq.NEXTVAL,
        'UPDATE',
        :NEW.student_id,
        :OLD.marks,
        :NEW.marks,
        SYSDATE
    );
END;
/
```

```sql
-- 6. DELETE Trigger
CREATE OR REPLACE TRIGGER trg_student_delete
AFTER DELETE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Log
    VALUES (
        log_seq.NEXTVAL,
        'DELETE',
        :OLD.student_id,
        :OLD.marks,
        NULL,
        SYSDATE
    );
END;
/

-- 7. Execute INSERT trigger
INSERT INTO Student
VALUES (101, 'Rahul', 85);

-- 8. Execute UPDATE trigger
UPDATE Student
SET marks = 95
WHERE student_id = 101;

-- 9. Execute DELETE trigger
DELETE FROM Student
WHERE student_id = 101;

COMMIT;

-- 10. Display trigger log records
SELECT * FROM Student_Log;

-- 11. Display triggers created
SELECT trigger_name, triggering_event, status
FROM USER_TRIGGERS
WHERE TABLE_NAME = 'STUDENT';
```

week - 7

```sql
-- 1. Create Student table
CREATE TABLE Student (
    student_id NUMBER(5) PRIMARY KEY,
    student_name VARCHAR2(30),
    marks NUMBER(5,2)
);

-- 2. Insert sample records
```

```
INSERT INTO Student VALUES (101, 'Rahul', 85);
INSERT INTO Student VALUES (102, 'Priya', 90);
INSERT INTO Student VALUES (103, 'Amit', 75);

COMMIT;

-- 3. Create a stored procedure to display all students
CREATE OR REPLACE PROCEDURE display_students
AS
BEGIN
    FOR rec IN (SELECT * FROM Student)
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            rec.student_id || ' ' ||
            rec.student_name || ' ' ||
            rec.marks
        );
    END LOOP;
END;
/

-- 4. Execute the stored procedure
SET SERVEROUTPUT ON;
EXECUTE display_students;

-- 5. Create a procedure to insert a student
CREATE OR REPLACE PROCEDURE add_student (
    p_id IN NUMBER,
    p_name IN VARCHAR2,
    p_marks IN NUMBER
)
AS
BEGIN
    INSERT INTO Student
    VALUES (p_id, p_name, p_marks);

    DBMS_OUTPUT.PUT_LINE('Student inserted successfully');
END;
/

-- 6. Execute the insert procedure
EXECUTE add_student(104, 'Sneha', 95);

COMMIT;

-- 7. Display all student records
SELECT * FROM Student;

-- 8. Modify the stored procedure
```

```
CREATE OR REPLACE PROCEDURE display_students
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Student Details:');

    FOR rec IN (SELECT * FROM Student ORDER BY student_id)
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || rec.student_id ||
            ', Name: ' || rec.student_name ||
            ', Marks: ' || rec.marks
        );
    END LOOP;
END;
/

-- 9. Execute the modified procedure
EXECUTE display_students;

-- 10. Display procedure details
SELECT OBJECT_NAME, STATUS
FROM USER_OBJECTS
WHERE OBJECT_TYPE = 'PROCEDURE'
AND OBJECT_NAME IN ('DISPLAY_STUDENTS', 'ADD_STUDENT');

week - 8
-- MINI PROJECT: STUDENT MANAGEMENT SYSTEM

-- 1. Create Department table
CREATE TABLE Department (
    dept_id NUMBER(5) PRIMARY KEY,
    dept_name VARCHAR2(30) UNIQUE NOT NULL
);

-- 2. Create Student table
CREATE TABLE Student (
    student_id NUMBER(5) PRIMARY KEY,
    student_name VARCHAR2(30) NOT NULL,
    age NUMBER(3) CHECK (age >= 17),
    gender VARCHAR2(10),
    marks NUMBER(5,2),
    dept_id NUMBER(5),
    FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
);

-- 3. Insert department records
INSERT INTO Department VALUES (10, 'CSE');
INSERT INTO Department VALUES (20, 'ECE');
INSERT INTO Department VALUES (30, 'EEE');
```

```sql
-- 4. Insert student records
INSERT INTO Student
VALUES (101, 'Rahul', 20, 'Male', 85, 10);

INSERT INTO Student
VALUES (102, 'Priya', 21, 'Female', 90, 20);

INSERT INTO Student
VALUES (103, 'Amit', 19, 'Male', 75, 10);

INSERT INTO Student
VALUES (104, 'Sneha', 20, 'Female', 95, 30);

COMMIT;

-- 5. Display all students
SELECT * FROM Student;

-- 6. Display all departments
SELECT * FROM Department;

-- 7. Search for a student by ID
SELECT *
FROM Student
WHERE student_id = 101;

-- 8. Display students with department names
SELECT S.student_id, S.student_name,
       S.age, S.marks, D.dept_name
FROM Student S
JOIN Department D
ON S.dept_id = D.dept_id;

-- 9. Update student marks
UPDATE Student
SET marks = 92
WHERE student_id = 102;

COMMIT;

-- 10. Display students scoring above 80
SELECT student_id, student_name, marks
FROM Student
WHERE marks > 80;

-- 11. Count total students
SELECT COUNT(*) AS total_students
FROM Student;
```

```sql
-- 12. Calculate average marks
SELECT AVG(marks) AS average_marks
FROM Student;

-- 13. Display department-wise student count
SELECT D.dept_name,
       COUNT(S.student_id) AS total_students
FROM Department D
LEFT JOIN Student S
ON D.dept_id = S.dept_id
GROUP BY D.dept_name;

-- 14. Create a view for student details
CREATE OR REPLACE VIEW Student_Details AS
SELECT S.student_id, S.student_name,
       S.age, S.gender, S.marks, D.dept_name
FROM Student S
JOIN Department D
ON S.dept_id = D.dept_id;

-- 15. Display the view
SELECT * FROM Student_Details;

-- 16. Delete a student record
DELETE FROM Studen WHERE student_id = 103;

COMMIT;

-- 17. Display final student records
SELECT * FROM Student;

-- 18. Display final student details through the view
SELECT * FROM Student_Details;
```

week - 9

```sql
-- 1. Create the original table
CREATE TABLE Student_Course (
    student_id NUMBER(5),
    student_name VARCHAR2(30),
    course_id NUMBER(5),
    course_name VARCHAR2(30),
    instructor_id NUMBER(5),
    instructor_name VARCHAR2(30),
    marks NUMBER(5,2),
    CONSTRAINT pk_sc PRIMARY KEY (student_id, course_id)
);

-- 2. Insert sample records
```

```
INSERT INTO Student_Course VALUES
(101, 'Rahul', 10, 'DBMS', 501, 'Kumar', 85);

INSERT INTO Student_Course VALUES
(101, 'Rahul', 20, 'Java', 502, 'Priya', 90);

INSERT INTO Student_Course VALUES
(102, 'Amit', 10, 'DBMS', 501, 'Kumar', 75);

INSERT INTO Student_Course VALUES
(103, 'Sneha', 20, 'Java', 502, 'Priya', 95);

COMMIT;

-- 3. Display the original table
SELECT * FROM Student_Course;

-- 4. Convert to 1NF
-- Each column contains a single value
-- The table already satisfies 1NF

-- 5. Convert to 2NF
-- Create Student table
CREATE TABLE Student (
    student_id NUMBER(5) PRIMARY KEY,
    student_name VARCHAR2(30)
);

INSERT INTO Student
SELECT DISTINCT student_id, student_name
FROM Student_Course;

-- Create Course table
CREATE TABLE Course (
    course_id NUMBER(5) PRIMARY KEY,
    course_name VARCHAR2(30),
    instructor_id NUMBER(5),
    instructor_name VARCHAR2(30)
);

INSERT INTO Course
SELECT DISTINCT course_id, course_name,
       instructor_id, instructor_name
FROM Student_Course;

-- Create Enrollment table
CREATE TABLE Enrollment (
    student_id NUMBER(5),
    course_id NUMBER(5),
```

```
    marks NUMBER(5,2),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Student(student_id),
    FOREIGN KEY (course_id) REFERENCES Course(course_id)
);

INSERT INTO Enrollment
SELECT student_id, course_id, marks
FROM Student_Course;

COMMIT;

-- 6. Display 2NF tables
SELECT * FROM Student;
SELECT * FROM Course;
SELECT * FROM Enrollment;

-- 7. Convert to 3NF
-- Separate instructor details into a new table
CREATE TABLE Instructor (
    instructor_id NUMBER(5) PRIMARY KEY,
    instructor_name VARCHAR2(30)
);

INSERT INTO Instructor
SELECT DISTINCT instructor_id, instructor_name
FROM Course;

-- Create normalized Course table
CREATE TABLE Course_3NF (
    course_id NUMBER(5) PRIMARY KEY,
    course_name VARCHAR2(30),
    instructor_id NUMBER(5),
    FOREIGN KEY (instructor_id)
        REFERENCES Instructor(instructor_id)
);

INSERT INTO Course_3NF
SELECT DISTINCT course_id, course_name, instructor_id
FROM Course;

COMMIT;

-- 8. Display 3NF tables
SELECT * FROM Student;
SELECT * FROM Instructor;
SELECT * FROM Course_3NF;
SELECT * FROM Enrollment;
```

```sql
-- 9. Display complete student enrollment details
SELECT S.student_id, S.student_name,
       C.course_name, I.instructor_name, E.marks
FROM Enrollment E
JOIN Student S ON E.student_id = S.student_id
JOIN Course_3NF C ON E.course_id = C.course_id
JOIN Instructor I ON C.instructor_id = I.instructor_id;
```

week - 10 (mongosh)

```
// 1. Create or switch to a database
use CollegeDB

// 2. Create a collection
db.createCollection("Student")

// 3. CREATE: Insert one student document
db.Student.insertOne({
    student_id: 101,
    name: "Rahul",
    age: 20,
    department: "CSE",
    marks: 85
})

// Insert multiple student documents
db.Student.insertMany([
    {
        student_id: 102,
        name: "Priya",
        age: 21,
        department: "ECE",
        marks: 90
    },
    {
        student_id: 103,
        name: "Amit",
        age: 19,
        department: "CSE",
        marks: 75
    },
    {
        student_id: 104,
        name: "Sneha",
        age: 20,
        department: "EEE",
        marks: 95
    }
])
```

```
// 4. READ: Display all student documents
db.Student.find()

// Display documents in a readable format
db.Student.find().pretty()

// Find a student by ID
db.Student.find({ student_id: 101 })

// Find students scoring above 80
db.Student.find({ marks: { $gt: 80 } })

// Display only name and marks
db.Student.find(
    {},
    { _id: 0, name: 1, marks: 1 }
)

// 5. UPDATE: Update one student's marks
db.Student.updateOne(
    { student_id: 101 },
    { $set: { marks: 92 } }
)

// Update multiple students' department
db.Student.updateMany(
    { department: "CSE" },
    { $set: { department: "Computer Science" } }
)

// Verify updated documents
db.Student.find()

// 6. DELETE: Delete one student
db.Student.deleteOne({ student_id: 103 })

// Delete students scoring below 80
db.Student.deleteMany({ marks: { $lt: 80 } })

// 7. Display final collection
db.Student.find()

// 8. Display collection names
show collections

// 9. Count remaining documents
db.Student.countDocuments({})
```

```
week - 10(Python)
pip install pymongo
# DBMS Lab: MongoDB CRUD Operations Using Python

from pymongo import MongoClient

# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/")

# Create database and collection
db = client["CollegeDB"]
students = db["Student"]

# CREATE: Insert student records
students.insert_many([
    {
        "student_id": 101,
        "name": "Rahul",
        "age": 20,
        "department": "CSE",
        "marks": 85
    },
    {
        "student_id": 102,
        "name": "Priya",
        "age": 21,
        "department": "ECE",
        "marks": 90
    },
    {
        "student_id": 103,
        "name": "Amit",
        "age": 19,
        "department": "CSE",
        "marks": 75
    }
])

print("Records inserted successfully.")

# READ: Display all students
print("\nAll Student Records:")
for student in students.find({}, {"_id": 0}):
    print(student)

# Search for a student
print("\nSearch Student with ID 101:")
print(students.find_one(
    {"student_id": 101},
```

```
    {"_id": 0}
))

# UPDATE: Update student marks
students.update_one(
    {"student_id": 101},
    {"$set": {"marks": 95}}
)

print("\nStudent marks updated successfully.")

# Display updated record
print(students.find_one(
    {"student_id": 101},
    {"_id": 0}
))

# DELETE: Delete a student record
students.delete_one({"student_id": 103})

print("\nStudent record deleted successfully.")

# Display final records
print("\nFinal Student Records:")
for student in students.find({}, {"_id": 0}):
    print(student)

# Close connection
client.close()
```

week - 11

```
# DBMS Lab: MongoDB Arrays, Count, Sort and Aggregate

from pymongo import MongoClient

# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/")

# Create database and collection
db = client["CollegeDB"]
students = db["Student"]

# Clear old records for repeatable output
students.delete_many({})

# 1. Insert documents containing arrays
students.insert_many([
    {
```

```
        "student_id": 101,
        "name": "Rahul",
        "department": "CSE",
        "marks": 85,
        "subjects": ["DBMS", "Python", "Java"]
    },
    {
        "student_id": 102,
        "name": "Priya",
        "department": "ECE",
        "marks": 92,
        "subjects": ["Python", "Networks", "DBMS"]
    },
    {
        "student_id": 103,
        "name": "Amit",
        "department": "CSE",
        "marks": 75,
        "subjects": ["Java", "Python"]
    },
    {
        "student_id": 104,
        "name": "Sneha",
        "department": "ECE",
        "marks": 95,
        "subjects": ["DBMS", "Networks"]
    }
])

print("1. All Student Records:")
for student in students.find({}, {"_id": 0}):
    print(student)

# 2. Array operation: Find students studying Python
print("\n2. Students studying Python:")
for student in students.find({"subjects": "Python"},{"_id": 0}):
    print(student["name"], student["subjects"])

# Find students whose subjects include both Python and DBMS
print("\n3. Students studying Python and DBMS:")
for student in students.find({"subjects": {"$all": ["Python",
"DBMS"]}},{"_id": 0, "name": 1, "subjects": 1}):
    print(student)

# 3. COUNT: Count all students
print("\n4. Total number of students:")
print(students.count_documents({}))

# Count students in CSE
print("\n5. Number of CSE students:")
```

```
print(students.count_documents({"department": "CSE"}))

# 4. SORT: Sort by marks in ascending order
print("\n6. Students sorted by marks (ascending):")
for student in students.find({},{"_id": 0, "name": 1, "marks":
1}).sort("marks", 1):
    print(student)

# Sort by marks in descending order
print("\n7. Students sorted by marks (descending):")
for student in students.find({},{"_id": 0, "name": 1, "marks":
1}).sort("marks", -1):
    print(student)

# 5. AGGREGATE: Calculate average marks
print("\n8. Average marks:")
result = list(students.aggregate([
    {
        "$group": {
            "_id": None,
            "average_marks": {"$avg": "$marks"}
        }
    }
]))
print(result)

# 6. Aggregate department-wise student count and average
print("\n9. Department-wise statistics:")
result = students.aggregate([
    {
        "$group": {
            "_id": "$department",
            "total_students": {"$sum": 1},
            "average_marks": {"$avg": "$marks"}
        }
    },
    {
        "$sort": {"_id": 1}
    }
])

for record in result:
    print(record)

# 7. Aggregate students with marks above 80
print("\n10. Students scoring above 80:")
result = students.aggregate([
    {"$match": {"marks": {"$gt": 80}}},
    {"$sort": {"marks": -1}},
    {"$project": {"_id": 0, "name": 1, "marks": 1}}
```

```
])

for record in result:
    print(record)

# Close MongoDB connection
client.close()
```

week - 12

```
pip install oracledb pymongo

# DBMS Lab: SQL and MongoDB Integration

import oracledb
from pymongo import MongoClient

# 1. Connect to Oracle SQL
sql_conn = oracledb.connect(
    user="system",
    password="your_password",
    dsn="localhost:1521/XEPDB1"
)
sql_cursor = sql_conn.cursor()

# 2. Connect to MongoDB
mongo_client = MongoClient("mongodb://localhost:27017/")
mongo_db = mongo_client["CollegeDB"]
student_collection = mongo_db["StudentDetails"]

# 3. Create SQL table if it does not exist
try:
    sql_cursor.execute("""
        CREATE TABLE Student (
            student_id NUMBER(5) PRIMARY KEY,
            student_name VARCHAR2(30),
            department VARCHAR2(20),
            marks NUMBER(5,2)
        )
    """)
    sql_conn.commit()
except oracledb.DatabaseError as e:
    error_obj = e.args[0]
    if error_obj.code != 955:
        raise

# 4. Insert student into SQL
student_id = 101
student_name = "Rahul"
department = "CSE"
marks = 85
```

```
sql_cursor.execute("""
    MERGE INTO Student s
    USING (
        SELECT :id AS student_id,
               :name AS student_name,
               :dept AS department,
               :marks AS marks
        FROM dual
    ) d
    ON (s.student_id = d.student_id)
    WHEN MATCHED THEN UPDATE SET
        s.student_name = d.student_name,
        s.department = d.department,
        s.marks = d.marks
    WHEN NOT MATCHED THEN INSERT
        (student_id, student_name, department, marks)
    VALUES
        (d.student_id, d.student_name, d.department, d.marks)
""", {
    "id": student_id,
    "name": student_name,
    "dept": department,
    "marks": marks
})

sql_conn.commit()
print("Student record saved in Oracle SQL.")

# 5. Insert additional details into MongoDB
student_collection.update_one(
    {"student_id": student_id},
    {
        "$set": {
            "student_id": student_id,
            "name": student_name,
            "hobbies": ["Reading", "Cricket"],
            "skills": ["Python", "SQL"]
        }
    },
    upsert=True
)

print("Additional details saved in MongoDB.")

# 6. Read student record from SQL
print("\nStudent details from Oracle SQL:")
sql_cursor.execute("""
    SELECT student_id, student_name, department, marks
    FROM Student
```

```
    WHERE student_id = :id
""", {"id": student_id})

for row in sql_cursor.fetchall():
    print(row)

# 7. Read additional details from MongoDB
print("\nAdditional details from MongoDB:")
document = student_collection.find_one(
    {"student_id": student_id},
    {"_id": 0}
)
print(document)

# 8. Update marks in Oracle SQL
sql_cursor.execute("""
    UPDATE Student
    SET marks = :marks
    WHERE student_id = :id
""", {"marks": 95, "id": student_id})

sql_conn.commit()
print("\nStudent marks updated in Oracle SQL.")

# 9. Display updated marks
sql_cursor.execute("""
    SELECT student_name, marks
    FROM Student
    WHERE student_id = :id
""", {"id": student_id})

print("Updated SQL record:", sql_cursor.fetchone())

# 10. Delete student from both databases
# Uncomment these lines if deletion is required:
# sql_cursor.execute(
#     "DELETE FROM Student WHERE student_id = :id",
#     {"id": student_id}
# )
# sql_conn.commit()
# student_collection.delete_one({"student_id": student_id})
# print("Student deleted from both databases.")

# 11. Close connections
sql_cursor.close()
sql_conn.close()
mongo_client.close()

print("\nIntegration program completed.")
```